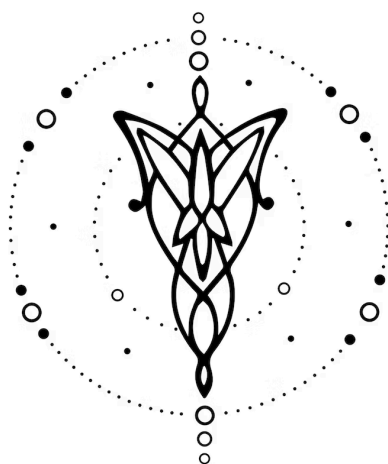




Especificación - Dataly

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	2
3. Dominio	2
4. Construcciones	3
5. Casos de Prueba	5
Ejemplo 1: Ingesta y limpieza de datos de ventas	7
Ejemplo 2: Enriquecimiento y agregación	8
Ejemplo 3: Ranking con funciones de ventana	8

1. Equipo

Nombre	Apellido	Legajo	E-mail
Santiago	Nogueira	64113	snogueira@itba.edu.ar
Santino	Pepe	64147	spepe@itba.edu.ar
Manuel	Araujo Feltrin	64090	maraujofeltrin@itba.edu.ar
Maria Victoria	Alvarez	63165	maralvarez@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Flex-Bison-Compiler](#).

3. Dominio

Desarrollar un lenguaje que permita definir y ejecutar pipelines de datos de manera declarativa, facilitando la construcción de procesos ETL y ELT. El lenguaje debe permitir declarar fuentes de datos en formato CSV, crear datasets a partir de dichas fuentes y aplicar transformaciones tabulares básicas como la selección de columnas y el filtrado de filas, para finalmente escribir los resultados en un único destino, ya sea un archivo CSV. A futuro, se prevé extender estas capacidades incorporando nuevas fuentes de datos comenzando por tablas SQL, continuando con Parquet, APIs o JSON. Asimismo, se ofrecerían funcionalidades como transformaciones avanzadas, tales como joins, agregaciones, funciones de ventana, creación de columnas derivadas y casts de tipos, junto con soporte para operaciones sobre fechas.

Para reducir la complejidad del proyecto, las transformaciones serán ejecutadas sobre datasets en memoria utilizando un backend simplificado (por ejemplo, scripts en Python con pandas¹). Asimismo, el lenguaje trabajará sobre un subconjunto limitado de tipos de datos (enteros, flotantes, booleanos, cadenas, fechas y decimales) y funciones básicas, dejando fuera casos avanzados como streaming en tiempo real.

Toda la ejecución se simula en el entorno local del compilador, sin conexión a infraestructuras externas ni orquestadores de pipelines, y los datos procesados existen únicamente dentro del programa resultante.

La implementación satisfactoria de este lenguaje permitiría a los usuarios diseñar, validar y documentar flujos de datos con garantías de tipado, esquemas consistentes y dependencias acíclicas, reduciendo errores y simplificando el trabajo de integración antes de desplegar estos procesos en plataformas de datos de mayor escala.

¹ <https://pandas.pydata.org/>

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

1. **Definir fuentes de datos (sources):** se podrán declarar una o varias fuentes de datos con nombre (en principio CSV y a futuro las declaradas en la especificación del dominio), ruta o conexión, y un esquema con columnas y tipos.
2. **Declarar datasets:** se podrán crear datasets a partir de las fuentes declaradas o de transformaciones intermedias.
3. **Aplicar transformaciones encadenadas:** cada dataset podrá ser transformado mediante operaciones declarativas que definen un flujo de procesamiento.
4. **Transformaciones soportadas (ordenadas por complejidad):**
 - **Selección de columnas (select)** con alias o expresiones derivadas.
 - **Filtros (filter)** con condiciones booleanas.
 - **Operaciones de ordenamiento (orderBy)** y límites (limit).
 - **Joins** entre dos datasets, con distintos tipos (inner, left, right, full, semi, anti).
 - **Creación de columnas (addColumn)** derivadas de expresiones.
 - **Cast de tipos** para conversión explícita.
 - **Agregaciones (groupBy)** con funciones como sum, avg, min, max, count.
 - **Funciones de ventana (window)** con partitionBy, orderBy y cálculos como row_number, rank, lead, lag, etc.
5. **Definir sinks (destinos):** los datasets transformados podrán ser escritos en distintos formatos (en principio CSV o SQL), con modos de escritura (overwrite, append, errorIfExists).

6. **Uso de parámetros (param):** se podrán declarar parámetros externos para rutas, fechas o filtros, con el fin de generar pipelines reutilizables y dinámicos.
7. **Soporte para funciones definidas por el usuario (udf):** se podrán declarar UDFs simples con firmas de tipos, para extender las transformaciones con lógica personalizada.
8. **Validaciones automáticas de esquemas y tipos:** el compilador verificará que todas las columnas existan, los tipos sean compatibles y los casts válidos.
9. **Verificación de dependencias (DAG):** el compilador garantizará que el grafo de transformaciones sea acíclico y que todos los datasets referenciados estén correctamente definidos.
10. **Soporte para null:** las operaciones respetarán la propagación de nulls, con funciones especiales como coalesce para controlarlos.
11. **Expresiones aritméticas básicas:** +, -, *, /, % sobre columnas numéricas.
12. **Expresiones lógicas básicas:** AND, OR, NOT, IS NULL, IS NOT NULL.
13. **Expresiones relacionales básicas:** <, >, =, ≠, ≤, ≥.
14. **Funciones de strings y fechas:** substring, length, concat, year, month, day, date_diff, regex_match.
15. **Manejo de JSON:** operaciones simples para extraer campos de columnas JSON.
16. **Visualización del pipeline:** se podrá exportar el DAG de dependencias como un grafo DOT/Graphviz.
17. **Exportar manifiesto del pipeline:** cada compilación generará un archivo JSON con metadatos de fuentes, transformaciones y sinks (lineage).
18. **Soporte de múltiples sinks en un mismo pipeline:** un programa podrá escribir en varios destinos en paralelo.
19. **Validación temprana de errores:** programas malformados, referencias a datasets inexistentes, ciclos en el DAG, columnas inexistentes o tipos incompatibles serán rechazados en compilación.

20. **Extensibilidad:** la sintaxis permitirá agregar futuras transformaciones, conectores de datos adicionales o funciones avanzadas de manera modular.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de aceptación:

1. Un programa que lea un CSV de ventas y lo escriba en Parquet aplicando un filtro y una selección de columnas.
2. Un programa que una (join) dos planillas de ventas por store_id.
3. Un programa que agregue por región y mes (groupBy) calculando sum(amount) y count(*).
4. Un programa que compute un ranking por región con window row_number() y lo exporte a CSV.
5. Un programa que declare parámetros (param data_dir, param run_date) y los use en rutas y filtros.
6. Un programa que defina y utilice una UDF tipada para normalizar un monto y valide su firma de tipos.
7. Un programa que realice cast explícito de string → date y int → decimal(12,2) antes de agrupar.
8. Un programa que produzca múltiples sinks (Parquet particionado por y,m y un reporte CSV resumido).
9. Un programa que valide null con coalesce y condiciones IS NULL / IS NOT NULL.
10. Un programa que genere el DAG de dependencias (DOT/Graphviz) y el manifest.json del pipeline.
11. Un programa que procese dos fuentes (CSV + SQL) y combine resultados en un único dataset final.
12. Un programa que aplique expresiones sobre fechas (year, month, date_diff) y strings (substring, regex_match).

Además, los siguientes casos de prueba de rechazo:

1. Un programa malformado (errores de sintaxis en declaraciones de source o transform).
2. Un programa que haga select de una columna inexistente en el dataset de entrada.
3. Un programa que realice groupBy y seleccione columnas que no son claves ni agregadas.
4. Un programa que invoque join sin condición on (producto cartesiano prohibido).
5. Un programa que contenga un ciclo en el DAG (una transformación depende indirectamente de sí misma).
6. Un programa que intente castear tipos incompatibles bajo modo estricto (p. ej., string a int con valores no numéricos).
7. Un programa que escriba (write) en un sink no declarado o en modo errorIfExists cuando el archivo ya existe.
8. Un programa que use una UDF con firma de tipos que no coincide con los argumentos provistos.
9. Un programa que haga join con columnas de tipos no compatibles sin cast explícito.
10. Un programa que referencie un dataset que no fue definido previamente (dependencia inexistente).

6. Ejemplos

Código 1: Ingesta y limpieza de datos de ventas

En el *Código 1* se crea un pipeline que lee un CSV de ventas, elimina las filas de montos negativos, deriva las columnas de año y mes y por último, lo escribe en formato Parquet:

```
/* Definir parámetros de entrada y salida */
param data_dir = "/data/in"
param out_dir = "/data/out"

/* Definir fuente de datos CSV con ventas */
source sales_src {
  format = "csv"
  path = "{data_dir}/sales.csv"
  header = "true"
}

/* Crear dataset a partir de la fuente de ventas */
dataset sales from sales_src

/* Transformar datos: filtrar, agregar columnas derivadas y seleccionar */
transform clean_sales = from sales use
  /* Filtrar solo ventas válidas (monto > 0 y no reembolsadas) */
  filter { amount > 0 and refunded = false }
  /* Agregar columnas de año y mes */
  withColumn { y = "2024", m = "01" }
  /* Seleccionar solo las columnas necesarias */
  select { id, store_id, y, m, amount }

/* Definir destino en formato Parquet */
sink fact_sales {
  format = "parquet"
  path = "{out_dir}/fact_sales"
  mode = "overwrite"
}

/* Escribir datos transformados al destino */
write clean_sales into fact_sales
```

Código 1: Implementación de procesamiento de datos.

En el **Código 1** se representa un pipeline ETL (Extract, Transform, Load) básico que demuestra las operaciones fundamentales de procesamiento de datos. El programa lee datos de ventas desde un archivo CSV, aplica transformaciones de limpieza (filtrado de ventas válidas y agregación de columnas de tiempo), y escribe el resultado en formato Parquet.

Código 2: Enriquecimiento y agregación

En el **Código 2** se crea un pipeline que une las ventas del ejemplo anterior con una segunda fuente y genera una reporte agrupando por región y mes:

```
/* Definir parámetros de entrada y salida */
param data_dir = "/data/in"
param out_dir = "/data/out"

/* Primera fuente: datos de ventas */
source sales_src {
  format = "csv"
  path = "{data_dir}/sales.csv"
  header = "true"
}

/* Crear dataset de ventas */
dataset sales from sales_src

/* Limpiar y transformar datos de ventas */
transform clean_sales = from sales use
  /* Filtrar ventas válidas */
  filter { amount > 0 and refunded = false }
  /* Agregar columnas de tiempo (valores fijos) */
  withColumn { y = "2024", m = "01" }
  /* Seleccionar columnas relevantes */
  select { id, store_id, y, m, amount }

/* Segunda fuente: datos de tiendas */
source stores_src {
  format = "csv"
  path = "{data_dir}/stores.csv"
  header = "true"
}

/* Crear dataset de tiendas */
dataset stores from stores_src

/* Transformación simplificada */
transform sales_by_region = from clean_sales use
  /* Seleccionar solo columnas de ventas */
```



```
select { store_id, y, m, amount }

/* Definir destino para reporte */
sink report {
  format = "csv"
  path = "{out_dir}/report.csv"
  mode = "overwrite"
}

/* Escribir datos procesados */
write sales_by_region into report
```

Código 2: Implementación de procesamiento de múltiples fuentes.

En el **Código 2** se extiende el concepto básico al procesar múltiples fuentes de datos (ventas y tiendas) en un solo pipeline. Aunque el compilador no soporta operaciones de join reales, el programa simula el concepto de combinar datos de diferentes fuentes. El pipeline procesa dos datasets independientes: uno de ventas (con limpieza y transformación) y otro de tiendas, luego crea una transformación que prepara los datos para análisis regional.

Código 3: Ranking con funciones de ventana

En el **Código 3** se crea un pipeline que guarda el ranking de regiones dependiendo de los ingresos usando una función de ventana (window) y se exporta el resultado:

```
/* Definir parámetros de entrada y salida */
param data_dir = "/data/in"
param out_dir = "/data/out"

/* Primera fuente: datos de ventas */
source sales_src {
  format = "csv"
  path = "{data_dir}/sales.csv"
  header = "true"
}

/* Crear dataset de ventas */
dataset sales from sales_src

/* Limpiar y transformar datos de ventas */
transform clean_sales = from sales use
```

```
/* Filtrar solo ventas válidas */
filter { amount > 0 and refunded = false }
/* Agregar columnas de tiempo (valores fijos) */
withColumn { y = "2024", m = "01" }
/* Seleccionar columnas necesarias */
select { id, store_id, y, m, amount }

/* Segunda fuente: datos de tiendas */
source stores_src {
  format = "csv"
  path = "{data_dir}/stores.csv"
  header = "true"
}

/* Crear dataset de tiendas */
dataset stores from stores_src

/* Primera transformación: preparar datos para análisis regional */
transform sales_by_region = from clean_sales use
  /* Seleccionar columnas relevantes para análisis */
  select { store_id, y, m, amount }

/* Segunda transformación: simular ranking */
transform rank_in_region = from sales_by_region use
  /* Agregar columna de ranking simulado */
  withColumn { rank = "1" }
  /* Seleccionar columnas finales incluyendo ranking */
  select { store_id, y, m, amount, rank }

/* Definir destino para reporte de ranking */
sink rank_report {
  format = "csv"
  path = "{out_dir}/rank.csv"
  mode = "overwrite"
}

/* Escribir datos con ranking al destino final */
write rank_in_region into rank_report
```

Código 3: Implementación de procesamiento de múltiples fuentes con múltiples transformaciones.

En el **Código 3** se representa el pipeline más complejo de los tres, demostrando un flujo de datos multi-etapa que simula operaciones avanzadas de análisis. El programa procesa las mismas fuentes de datos (ventas y tiendas) pero aplica múltiples transformaciones secuenciales: primero limpia los datos de ventas, luego prepara un dataset para análisis regional, y finalmente simula una operación de ranking agregando una columna de ranking.